

Intelligent Photo Puzzle Solver Using Computer Vision Techniques

Jyotsana Parkhedkar - 001661399

Harshini Narra - 001645642

Prathamesh Kale

Rishikesh Sirisilla

CSI 471/571 – Introduction to Computer Vision

Instructor: Dr. Xin Li

Fall 2025

1 Introduction

This project presents the design and implementation of an **Interactive Photo Puzzle Solver** developed in MATLAB. The system takes an input image, divides it into an $N \times N$ grid of tiles, shuffles the tiles, and enables the user to solve the puzzle using a drag-and-drop graphical user interface (GUI).

What makes this project interesting is the integration of several **computer vision algorithms** to assist the user. Specifically, the system computes

- edge compatibility between tiles,
- color histogram similarity, and
- SURF (Speeded-Up Robust Features) matching.

These cues are combined to generate intelligent neighbor suggestions, helping the user solve the puzzle more efficiently.

This project aligns directly with the learning outcomes of CSI 471/571, which emphasize edge detection, color processing, feature extraction, feature matching, and practical programming in MATLAB. It touches both low-level vision (edges, filtering) and mid-level vision (feature matching and registration), and it demonstrates how these techniques can be embedded in an interactive application.

2 Motivation and Problem Definition

Solving a jigsaw puzzle is an intuitive example of image understanding, involving recognition of edges, colors, textures, and spatial relationships. Automated or assisted puzzle solving has real-world relevance in

- forensics (shredded document reconstruction),
- archaeology (artifact reassembly),
- photo mosaicing and image stitching, and
- robotics assembly tasks.

The main problem addressed in this project can be stated as follows:

Given a shuffled grid of image tiles, how can we measure how close the arrangement is to the original image and provide accurate suggestions for which tiles should be adjacent?

To address this, we implemented:

- a drag-and-drop GUI for user interaction,
- a tile similarity engine using computer vision algorithms,

- a progress indicator showing how far the puzzle is from the solution, and
- a completion detector that displays a “Game Over” message when the puzzle is solved.

3 System Design and Architecture

The system consists of four main components: the GUI, the tiling and shuffling module, the intelligent neighbor suggestion system, and the progress/completion logic.

3.1 Graphical User Interface

The GUI is implemented using MATLAB’s figure and `uicontrol` framework. It includes:

- a large left panel displaying the puzzle,
- a right-side control panel containing the grid-size selector, buttons for loading, shuffling, and neighbor suggestion,
- a dynamic progress indicator, and
- a textual panel for instructions and suggestion details.

To ensure readability and aesthetics, the interface uses larger fonts, bold headers, color-coded buttons, and a soft background palette. This makes the system presentation-ready and pleasant to interact with.

3.2 Image Tiling and Shuffling

Once the user selects an image, the system:

1. resizes the image to manageable dimensions,
2. divides it into an $N \times N$ grid of tiles by slicing rows and columns,
3. stores the tiles in a cell array, and
4. shuffles tile indices using MATLAB’s `randperm` function.

The `randperm` function internally performs a Fisher–Yates shuffle, producing a uniform random permutation of tile indices. A matrix `currentOrder` tracks which tile index is placed at each grid location. The puzzle state is solved exactly when

$$\text{currentOrder} = \text{reshape}(1:N^2, N, N).$$

3.3 Intelligent Neighbor Suggestion System

The core of the project is the computation of tile similarity. For each tile, the system precomputes three descriptors.

3.3.1 Edge Descriptor

The tile is converted to grayscale, and pixel values along the top, bottom, left, and right borders are extracted and concatenated into a one-dimensional vector. The similarity between two tiles is measured using the Euclidean (L2) distance:

$$d_{\text{edge}}(i, j) = \|e_i - e_j\|_2,$$

where e_i and e_j are the edge vectors of tiles i and j . Smaller values indicate more compatible borders.

3.3.2 Color Histogram Descriptor

Each tile is represented by a 16-bin histogram for each color channel (R, G, B). The histograms are concatenated and L2-normalized. Similarity between tiles i and j is measured using cosine similarity:

$$\text{sim}_{\text{color}}(i, j) = \frac{h_i^\top h_j}{\|h_i\|_2 \|h_j\|_2},$$

and converted to a distance $d_{\text{color}}(i, j) = 1 - \text{sim}_{\text{color}}(i, j)$.

3.3.3 SURF Descriptor

For each tile, the system detects SURF keypoints on the grayscale image and extracts local feature descriptors. Given descriptor sets D_i and D_j for tiles i and j , the function `matchFeatures` is used to count the number of matched keypoints. The SURF-based similarity score is

$$s_{\text{SURF}}(i, j) = \# \text{matches}(D_i, D_j),$$

where higher scores imply stronger texture similarity.

3.3.4 Combined Ranking

To suggest neighbors for a selected tile k , the system computes edge distance, color distance, and SURF score to all other tiles. Each quantity is normalized to the range $[0, 1]$ and combined into a single cost:

$$C(j) = \alpha \tilde{d}_{\text{edge}}(k, j) + \beta \tilde{d}_{\text{color}}(k, j) + \gamma \tilde{s}_{\text{SURF}}(k, j),$$

where tildes denote normalized quantities and α, β, γ are weights (set equal in this project). Tiles with the smallest cost are highlighted in the GUI as recommended neighbors.

3.4 Progress Measurement and Completion Detection

To provide feedback on how close the user is to the correct solution, the system computes:

- the number of tiles currently in their correct locations, and
- a lower bound on the number of swaps needed to solve the puzzle.

Let M be the number of misplaced tiles. Each swap can fix at most two tiles, so a lower bound on the number of swaps is

$$\text{minSwaps} = \left\lceil \frac{M}{2} \right\rceil.$$

A progress message of the form

“Progress: at least minSwaps swaps remaining (X/N tiles correct).”

is shown on the interface. When all tiles are in the correct positions, the system displays a “Game Over: puzzle solved” message.

4 Experimental Setup and Results

Experiments were conducted on puzzles of sizes 3×3 , 4×4 , and 5×5 , using natural scene images, portraits, and textured patterns. MATLAB R2023b with the Computer Vision Toolbox was used.

Two baselines were compared:

1. edge-only similarity, and
2. the combined edge + color + SURF similarity.

We evaluated:

- the accuracy of suggested neighbors,
- the number of user interactions required to solve the puzzle, and
- system responsiveness.

On a 3×3 puzzle, edge-only matching produced many incorrect suggestions in uniform regions, while the combined method achieved significantly higher neighbor accuracy, especially in images with strong color and texture cues. Informal user testing indicated that puzzles were solved roughly two to three times faster with suggestions enabled compared to manual trial-and-error.

5 Conclusion and Future Work

This project demonstrates how fundamental computer vision concepts—edges, color histograms, and feature matching—can be integrated into an interactive application. We developed a visually appealing MATLAB GUI and added an intelligent suggestion system that helps users solve photo puzzles more efficiently.

Key lessons learned include:

- feature extraction is crucial for representing local image patches,
- combining multiple cues is more robust than relying on a single metric, and
- user interface design strongly influences perceived system quality.

Potential extensions include:

- using deep learning embeddings (e.g., CNN features) for more discriminative patch similarity,
- handling rotated or flipped tiles,
- implementing global puzzle optimization using graph-based methods, and
- supporting irregular, jigsaw-shaped pieces rather than square tiles.

Overall, the project provided practical experience with computer vision algorithms and strengthened the connection between the theoretical material in CSI 471/571 and a real, interactive system.

References

- [1] H. Bay, T. Tuytelaars, and L. Van Gool, “SURF: Speeded-Up Robust Features,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2006.
- [2] D. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [3] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [4] T. Cho, S. Avidan, and W. Freeman, “A Probabilistic Image Jigsaw Puzzle Solver,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2010.
- [5] M. Pomeranz, R. Shemesh, and O. Ben-Shahar, “Fully Automatic Puzzle Solving Using Computer Vision,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2011.